4.4 Types of programming language

When you are writing computer programs you will make use of specific software that allows you to write, edit, test and run the programs you write.

Programming languages can be categorised as **high-level** and **low-level**.

High-level

A high-level language uses human-language style words. These could be English words that we recognise and use day-to-day. For example, `if`, `while`, `output`, `print`, `input`; as people we understand these, but they are also high-level language instructions. You will be learning to use a high-level language, for example, Python, Java and VB.NET.

Here are some examples of high-level language statements:

Python:

```
print("This is a high-level language statement")
```

VB.NET:

```
Dim x as integer
x = Console.readline()
```

Java:

```
num1 = 10;
num2 = 5;
num3 = num1 + num2;
```

A program that is written in a high-level language is considered **portable**; you can write a program on one computer, and run it on a different computer.

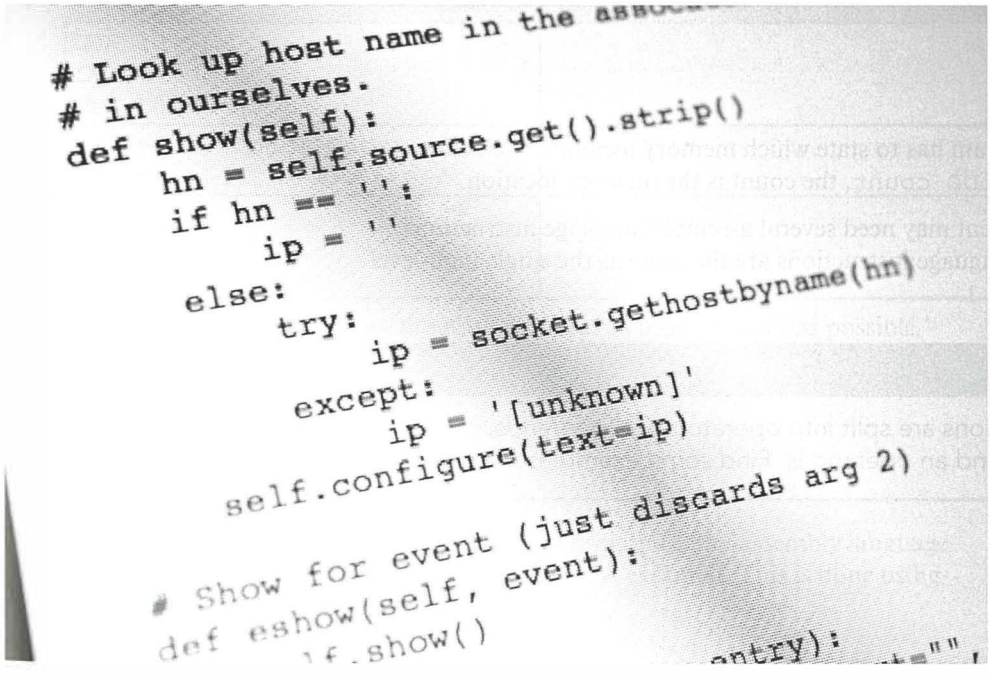


Figure 4.9: Code written in a high-level language

KEY WORDS

high-level language: a type of programming language that uses English-like commands.

low-level language: a language closer to the machine's language, this could be assembly language or binary code.

portable: a program that can be run on different types and manufacturers of computers.

Low-level

There are two main types of low-level language, **machine code** and **assembly language**.

A computer executes machine code, this is in binary (1s and 0s). If you write a program in a high-level language it needs to be converted into machine code before it can executed. Different computers may have different machine code (for example, depending on the manufacturer). This means that machine code is **non portable**; you can write a program in machine code and it may not run on a different computer.

An example of machine code is: `01100110`

Assembly language is an in-between stage. It uses **mnemonics** to represent code, for example, to store data in a memory location it may use the mneumonic `STO`. This is a mid-way stage. Each high-level language statement is converted into assembly language instructions, which are then converted into machine code.



Figure 4.10: Machine code

Examples of assembly language are:

```
LDD count
ADD 1
STO count
```

In assembly language, the program has to state which memory locations are being used. In the previous example, `LDD count`, the count is the memory location.

One high-level language statement may need several assembly language instructions. The previous three assembly language instructions are the same as the single high-level language `count = count + 1`.

ACTIVITY 4.2

Assembly language instructions are split into operators and operands. Find out what an operator and an operand is. Find some examples of each.

KEY WORDS

machine code: binary code, an example of a low-level language.

assembly language: code written in mnemonics that allows direct manipulation of the hardware. It must be converted into binary code to run.

non portable: a program that cannot be run on different types and manufacturers of computers.

mnemonic: instruction code used in assembly language.

Choosing a language

High- and low-level languages have their own benefits and drawbacks. Some of these are shown in Table 4.2.

High-level language	Low-level language
Easier for users (humans) to understand, read, write, amend.	More difficult for users to understand, read, write and amend.
Easier for users (humans) to debug code.	Harder for users to debug code.
Portable, machine independent.	Not portable, machine dependent.
It must be converted into a low-level language before it can be run.	Machine code does not need converting. Assembly language must be assembled before execution, but this is faster than from a high-level language.
One statement can represent many low-level instructions.	Several instructions are needed for each high-level language statement.
Cannot directly manipulate the hardware.	Can directly manipulate the hardware, e.g. writing to specific memory locations. This can make the program more efficient in terms of speed and memory usage.

Table 4.2: Benefits and drawbacks of high- and low-level languages

Questions

7 Describe what is meant by a high-level language.

8 Three examples of program code are given. Identify whether each example is a high-level language, assembly language, or machine code.

Program code	High-level, assembly or machine code

`LDX 10`
`INC`
`STO 10`	

`value1 = input("Enter value")` `print(value1 + 1)`	

`01100011`
`11000011`
`10101010`	

9 Harry needs to write a utility program that uses as little memory as possible. Give the most appropriate type of programming language Harry should use and justify your choice.

4.5 Translators

In Section 4.4 you learnt that high-level language programs, and assembly language programs, need to be converted to machine code to be executed. This is done using a **translator**.

KEY WORD

translator: a type of software that converts code written in one programming language into another, usually a high-level language into a low-level language.

Assembler

If you have a program written in assembly language, then an **assembler** is used to convert the instructions into machine code for the processor the code is being run on.

Interpreters and compilers

If you have written a program in a high-level language then you can use an **interpreter** or a **compiler** to translate the instructions.

An interpreter:

- reads one line of the high-level language code
- checks that it is syntactically correct:
  - if it is not, it stops and reports an error to the user
  - If it is, it executes the statement and moves to the next.

Interpreters are most useful when writing the program, because it will stop at an error, allow you to correct it, and then continue executing from the same position.

Interpreters are not as useful for whole-system testing, or when the program is complete. This is because all of the code needs to be interpreted every time the program is run, even if there are no errors.

A compiler checks all the code, by going through one-line after the other. If there are any syntax errors, they are all reported to the user and the program is not executed. If there are no errors, then an **executable file** is created. An executable file uses intermediate or object code derived from the high-level language code. The actual program code is not included so if someone has the executable file, they cannot view or amend the original code. The executable file can be run without having to re-translate the code.

Compilers are useful when you have finished the program or need to test a working element repeatedly. This is because the code does not need to be translated each time, the executable file is run repeatedly.

Table 4.3 shows the comparisons of interpreters and compilers.

KEY WORDS

assembler: converts assembly language into machine code.

interpreter: a translator that converts a high-level language into a low-level language. It checks one line of code and then executes it before checking the next.

compiler: a translator that converts a high-level language into a low-level language. It checks all code before running the program.

executable file: a file produced by a compiler that can be run independent of translator software and does not include the source code.

Interpreter	Compiler
Translates one line of code into machine code and then executes (runs) it.	Translates all lines of code into machine code, before executing the program.
Reports a syntax error as soon as it is picked up, and stops the program running until corrected.	Reports all syntax errors at the same time, the program is not run until all errors are corrected.
Useful when writing a program.	Useful when a program has been finished and is ready for testing or distribution.
Code needs to be re-translated each time the program is run.	Code does not need re-translating each time the program is run.
Does not produce an executable file.	Produces an executable file.
Source code is required to run the program.	Source code is not required to run the program, only the executable file.

(continued)

Interpreter	Compiler
Interpreter software is required to run the program.	No other software is required to run the executable file.
You can test part of a program without completing it all.	You need to have finished a section of code before testing it.

Table 4.3: Comparison of interpreters and compilers

Examples:

Saria is writing a computer game program that allows a character to move through different worlds collecting coins.

Which translator should Saria use when writing the code?

- It is more appropriate to use an interpreter. This will allow her to see when an error is identified, correct it and then continue running the program from the same position.
- A compiler can be used, but then all the errors would appear at the start and she would need to correct all of them before seeing how any of her program runs.
- An interpreter is considered more appropriate, but you could justify the use of a compiler in this situation.

Which translator should Saria use once she has finished programming the computer game and wants to sell it to users?

- A compiler should be used, because she can produce an executable file. There should be no syntax errors at this point, so correcting the code line-by-line is not required. The executable file can be distributed to people and they can then run the program without need to retranslate the instructions every time. The executable file also means that no-one will be able to see her code, so it is protected.
- An interpreter would not be an appropriate choice here, because the end user would need the interpreter software and the source code, and will need to wait for it to be interpreted each time. It also means the users can access the source code, and then copy it or change it.

ACTIVITY 4.3

Write a computer program in a language that uses an interpreter, and one that uses a compiler. You might need to use a different programming language for each one. For example, Python usually has interpreters, whilst VB.NET and Java usually have compilers. Which translator did you prefer to use?

Peer Assessment

Discuss the reasons why you preferred the translator you chose. Did anyone give you some reasons that you hadn't thought of? Can you expand your own answer any further?

SKILLS FOCUS 4.1

JUSTIFYING CHOICE

In an exam you may need to make a decision and then justify your choice. To answer this type question you will need to consider the context of the question and how each of the possible choices will impact the context. When you are justifying your decision you need to relate every reason back to the scenario.

Consider the choice of translator.

- An interpreter is most useful during the development of a program because:
  - you can fix errors in real time
  - the program will stop at each error to allow you to fix it
  - you can run small sections of the program when incomplete.
- A compiler is most useful when a program is complete because:
  - it produces a .exe file
  - the program does not need to be interpreted each time it is run
  - the program can be distributed without the source code.

Questions

Saria is writing a computer program.

1 Why is it more appropriate for Saria to use an interpreter while writing a program?

2 Why is it more appropriate for Saria to use a compiler when the program is complete?

4.6 Integrated Development Environment (IDE)

When you write a computer program you will most likely use an **Integrated Development Environment (IDE)**. This is a piece of software that includes an **editor** for you to enter and amend your code. This editor may include other features such as:

- **Auto-completion**; you start typing a command and it suggests options for you to select the correct one.
- **Auto-correction**; if you spell a command incorrect it changes it to what you meant to type.
- **Prettyprint**; this changes the colour of words, e.g. commands and identifiers. This helps you spot key terms and whether you have entered them correctly or not.
- Block minimising; you can make sections of code, e.g. procedures, minimised so there is less code to read through.

The IDE will also include the relevant translator, that you can select to translate the code. It will also provide a **run-time environment**; this is where the outputs from the executed code is displayed, it can be a command line or GUI depending on your language and the code entered.

KEY WORDS

Integrated Development Environment (IDE): a piece of software that allows a user to write, test and run program code.

editor: a feature of an IDE that allows the user to enter and amend program code.

auto-completion: a feature of an editor that gives the user options when they start typing a command.

auto-correction: a feature of an editor that identifies spelling errors and changes them.

prettyprint: a feature of an editor that changes the colour of text, for example, to highlight key words.

run-time environment: a feature of an IDE that allows a program to be run and lets the user interact with the program.

An IDE can also provide debugging features, or error diagnostics, these help you find the problems in your code. Some examples are:

- Break points; the user sets these points and the code stops running on that line. This lets you check the values and output from the code to that point.
- Variable watch window; the values of the variables are displayed while the code is running, so they can be checked whether they are correct.
- Stepping; the program will run one line at a time, and the user has to tell the program to move on to the next line, so it can be checked line by line.

You can write and run programs without an IDE. You can write your code in a basic text file without any formatting, and then run it (usually) through a command line interface by entering the correct instructions. An IDE is much more user friendly, with lots of tools to help you write, amend and correct your code.

COMPUTER SCIENCE IN CONTEXT

You may not always get a choice of whether to use an interpreter or a compiler. If you use an IDE (Integrated Development Environment) it will probably come with one, or the other. If your IDE has an interpreter and you want to make an executable then you might need to download another IDE to compile it.

ACTIVITY 4.4

Open up the IDE that you use to write programs. Identify the features that are mentioned in this chapter. Are there any different ones? If so, find out what these do.

Questions

10 Name **three** types of translator.

11 Nasrat is writing a program in a high-level language that she wants to sell to make money. Identify when Nasrat could use an interpreter, and when she could use a compiler. Justify your decisions.

12 Describe **two** features of an IDE that help the user to test the program they have written.

SELF ASSESSMENT

Question 11 required you to justify your choice. Did you give generic points, or did you relate each point you made to the scenario in the question? When a context is given, always try and go back to the question each time.

REFLECTION

After completing Activity 4.4, it can be helpful to think about how you approached it and ways you could improve your research skills for future activities.

1 Discuss how you approached the investigation of the features. Did you test out the features, or search the internet to find out what they did? Do you think your approach was the most appropriate? Would you try a different way if the activity was repeated?

2 Discuss how this activity has helped you to identify the features. Does practically using them help you to remember them?